RDBMS 名	MySQL8.0
データベース名	spring_sample
ユーザ名	spring_sample_user
パスワード	trainocate

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	255	−	−
3	価格	price	INT	−	−	−

商品番号	商品名	価格
1	スマートテレビ	30000
2	ゲーミングモニター	20000
3	ノートパソコン	50000
4	2 ドア冷蔵庫	50000
5	コンパクト冷蔵庫	15000
6	ワインクーラー	30000
7	全自動洗濯機	25000
8	乾燥機能付き洗濯機	40000
9	洗濯乾燥機	60000

要素	JDBC	Spring Data JPA
外部ライブラリの依存	JDBC ドライバー（mysql-connector-j.jar）を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。
ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。
冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。
例外処理の必要性	DriverManager.getConnectionメソッドが接続失敗時にSQLExceptionをスローするため、追加の例外処理コードが必要。	RuntimeExceptionを使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。
リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。

設定項目	意味
spring.datasource.url	データベースの URL(サーバー名/データベース名が含まれます)
spring.datasource.username	データベースに接続するユーザ名
spring.datasource.password	データベースに接続するためのパスワード
spring.jpa.show-sql=true	生成したSQL 文をログで確認可能

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	30	NOT NULL	−
3	価格	price	INT	−	NOT NULL	−

No	論理名称	物理名称	データ型	制約	備考
1	社員番号	emp_id	INT	PRIMARY KEY	−
2	パスワード	password	VARCHAR(15)	NOT NULL	−
3	社員名	emp_name	VARCHAR(30)	NOT NULL	−

emp_id	password	emp_name
123456	pass1234	山田太郎
234567	pass2345	佐藤花子
345678	pass3456	鈴木一郎

No	論理名称	物理名称	データ型	制約	備考
1	カテゴリID	id	INT	PRIMARY KEY	-
2	カテゴリ名	name	VARCHAR(30)	NOT NULL	-

カテゴリID	カテゴリ名
1	テレビ・モニター
2	コンピュータ
3	冷蔵庫
4	洗濯機

No	論理名称	物理名称	データ型	桁数	制約	備考
1	商品番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	商品名	name	VARCHAR	30	–	–
3	価格	price	INT	–	–	–
4	カテゴリID	category_id	INT		FOREIGN KEY	(Category テーブルのid 列を参照)

商品番号	商品名	価格	カテゴリID
1	スマートテレビ	30000	1
2	ゲーミングモニター	20000	1
3	ノートパソコン	50000	2
4	2 ドア冷蔵庫	50000	3
5	コンパクト冷蔵庫	15000	3
6	ワインクーラー	30000	3
7	全自動洗濯機	25000	4
8	乾燥機能付き洗濯機	40000	4
9	洗濯乾燥機	60000	4

4	カテゴリID	category_id	INT		FOREIGN KEY	(Category テーブルのid 列を参照)

列の物理名	列の論理名
id	商品番号
name	商品名
price	価格
description	商品説明
brand	ブランド名
sku	SKU コード
category_id	カテゴリID
price	価格

メソッド	目的	JPQL クエリ
findTotalSalesByCategory()	カテゴリごとの売上合計を取得	SUM(price) GROUP BY category.name
findProductCountByCategory()	カテゴリごとの商品数を取得	COUNT(*) GROUP BY category.name
findCategoriesByMinSales(Long minSales)	価格の合計が指定値以上のカテゴリを取得	HAVING SUM(price) >=minSales

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	−	−	−
3	社員名	name	VARCHAR	−	−	−
4	給与	salary	INT	−	−	

id	password	name	salary
1001	password	山田太郎	230000
1002	password	鈴木一郎	230000
1003	password	田中花子	300000
1004	password	山田次郎	400000
1005	password	高橋美智子	500000
1006	password	平恵	270000
1007	password	佐々木舞	330000
1008	password	中村翔	350000

対象	チェック内容	出力メッセージ
社員番号	①未入力時 ②4 桁未満の社員番号入力時	社員番号を入力してください 4 桁以上の社員番号を入力してください
パスワード	①未入力時	パスワードを入力してください

対象	チェック内容	出力メッセージ
社員番号 （アノテーション削除）	①未入力時 ②4 桁未満の社員番号入力時	社員番号を入力してください 4 桁以上の社員番号を入力してください
パスワード	①未入力時	パスワードを入力してください
社員名	①未入力時	名前を入力してください
給与	①未入力時 ②入力値が1 未満の場合	給与を入力してください 1 以上の数値を入力してください

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	−	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	−	−	−
3	社員名	name	VARCHAR	−	−	−

4	給与	salary	INT	−	−	
5	勤務地番号	location_id	INT		FOREIGN KEY	外部キー

ID	password	name	salary	location_id
1001	password	山田太郎	230000	1
1002	password	鈴木一郎	230000	2
1003	password	田中花子	300000	3
1004	password	山田次郎	400000	1
1005	password	高橋美智子	500000	1
1006	password	平恵	270000	5
1007	password	佐々木舞	330000	4
1008	password	中村翔	350000	3

No	論理名称	物理名称	データ型	桁数	制約	備考
1	勤務地番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	勤務地名	name	VARCHAR	–	–	–

id	name
1	tokyo
2	osaka
3	fukuoka
4	sapporo
5	okinawa

小問	内容
a	入力画面へ遷移するController メソッドの作成
b	location テーブルのリストを使って、<select> メニューを表示
c	EmpForm にlocationId を追加
d	登録処理の実装（save 処理）
e	外部キーの登録時に必要なエンティティ側の設定を確認

フィールド	説明
location（Location 型）	外部テーブルと結びつけるためのオブジェクト。SELECT では自動でJOIN してくれるが、登録・更新には使わない。
locationId（Integer 型）	フォームから渡された実際の整数値を使って登録・更新を行うためのフィールド。DB のカラムに直接反映される。
insertable = false, updatable = false	location（参照オブジェクト）を登録や更新に使わないようにHibernate に明示するための設定。

入力項目	内容
給与	type=″range″ で、給与を 200,000～500,000 の範囲から選べるようにしている
勤務地	DB のLocation テーブルから name を取得し、セレクトメニューに表示している

内容	詳細
URL	/findHighSalary
HTTP メソッド	GET
引数	int salary, String locationName, Model model
使用するリポジトリメソッド	findHighSalaryByLocation()（演習6.6.c で実装済み）
HTML ファイル	検索結果は emp_list.html に表示
Model への追加	取得したリストを "empList" のキー名で格納

ファイル名	格納場所
bootstrap.min.css	src/main/resources/static/css/
bootstrap.bundle.min.js	src/main/resources/static/js/

修正ポイント	内容
<table>	border="1" を削除し、Bootstrap の .table クラスを追加する
<thead>	ヘッダー行（見出し）を囲むタグとして追加する
<tbody>	データ行（繰り返し部分）を囲むタグとして追加する

クラス名	効果
table	Bootstrap 標準のテーブルデザイン
table-bordered	枠線をつける
table-hover	ホバー時に行の背景色が変わる